US Patent & Trademark Office
Patent Public Search | Text View

United States Patent 12400672
Kind Code B2
Date of Patent August 26, 2025
Inventor(s) O'Malley; Tom et al.

Generalized automatic speech recognition for joint acoustic echo cancellation, speech enhancement, and voice separation

Abstract

A method for training a generalized automatic speech recognition model for joint acoustic echo cancellation, speech enhancement, and voice separation includes receiving a plurality of training utterances paired with corresponding training contextual signals. The training contextual signals include a training contextual noise signal including noise prior to the corresponding training utterance, a training reference audio signal, and a training speaker vector including voice characteristics of a target speaker that spoke the corresponding training utterance. The operations also include training, using a contextual signal dropout strategy, a contextual frontend processing model on the training utterances to learn how to predict enhanced speech features. Here, the contextual signal dropout strategy uses a predetermined probability to drop out each of the training contextual signals during training of the contextual frontend processing model.

Inventors: **O'Malley; Tom (Mountain View, CA), Wang; Quan (Hoboken, NJ), Narayanan; Arun (Santa Clara, CA)**

Applicant: **Google LLC** (Mountain View, CA)

Family ID: **1000008780400**

Assignee: **Google LLC (Mountain View, CA)**

Appl. No.: **18/171368**

Filed: **February 19, 2023**

Prior Publication Data

Document Identifier	Publication Date
US 20230298609 A1	Sep. 21, 2023

Related U.S. Application Data

us-provisional-application US 63269629 20220320

Publication Classification

Int. Cl.: **G10L15/20** (20060101); **G10L15/06** (20130101); **G10L21/0208** (20130101); **G10L21/0272** (20130101)

U.S. Cl.:

CPC **G10L21/0208** (20130101); **G10L15/063** (20130101); G10L2021/02082 (20130101)

Field of Classification Search

CPC: G10L (21/0208); G10L (15/063); G10L (2021/02082); G10L (21/0272); G10L (15/16); G10L (15/20)

USPC: 704/231; 704/232; 704/233; 704/238; 704/239; 704/246

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
2011/0300806	12/2010	Lindahl	455/63.1	G10L 21/0208
2015/0294670	12/2014	Roblek	704/232	G10L 17/18

OTHER PUBLICATIONS

International Search Report and Written Opinion for the related Application No. PCT/US2023/062886, dated May 4, 2023, 29 pages. cited by applicant

Rajeev Iukhye et al: “Closing the Gap between Single-User and Multi-User VoiceFilter-Lite”, Arxiv.Org, Cornell University Library, 201 Olin Library Cornell University Ithaca, NY 14853, Feb. 24, 2022 (Feb. 24, 2022), XP091164401, section 3.3, first paragraph, l.ast sentence, 7 pages. cited by applicant

O'Malley Tom et al: “A Conformer-Based ASR Frontend for Joint Acoustic Echo Cancellation, Speech Enhancement and Speech Separation”, 2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU), IEEE, Dec. 13, 2021 (Dec. 13, 2021), pp. 304-311, XP034076936 , DOI: 10.1109/ASRU51503. 2021.9687942 retrieved on Jan. 20, 2022] p. 304, penu1timate and 1ast paragraphs bridging over to p. 305, figures 1,2, sections 2.2.1, 2.2.2, 2.2.3, 2.3, 2.4.1, 2.4.2, 8 pages. cited by applicant

Primary Examiner: Monikang; George C

Attorney, Agent or Firm: Honigman LLP

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS (1) This U.S. patent application claims priority under 35 U.S.C. § 119(e) to U.S. Provisional Application 63/269,629, filed on Mar. 20, 2022. The disclosure of this prior application is considered part of the disclosure of this application and is hereby incorporated by reference in its entirety.

TECHNICAL FIELD

(1) This disclosure relates to generalized automatic speech recognition for joint acoustic echo cancellation, speech enhancement, and voice separation.

BACKGROUND

(2) Robustness of automatic speech recognition (ASR) systems has significantly improved over the years with the advent of neural network-based end-to-end models, large-scale training data, and improved strategies for augmenting training data. Nevertheless, various conditions such as echo, harsher background noise, and competing speech significantly deteriorate performance of ASR systems. A joint ASR model may be trained to handle these conditions. However, in use, the joint ASR model may not encounter all conditions occurring at the same time. Accordingly, training the joint ASR model with all conditions present is not practical.

SUMMARY

(3) One aspect of the disclosure provides a computer-implemented method for training a generalized automatic speech recognition model for joint echo cancellation, speech enhancement, and voice separation that, when executed on data processing hardware, causes the data processing hardware to perform operations. The operations include receiving a plurality of training utterances paired with corresponding training contextual signals. The training contextual signals include a training contextual noise signal including noise prior to the corresponding training utterance, a training reference audio signal, and a training speaker vector including voice characteristics of a target speaker that spoke the corresponding training utterance. The operations also include training, using a contextual signal dropout strategy, a contextual frontend processing model on the training utterances to learn how to predict enhanced speech features. Here, the contextual signal dropout strategy uses a predetermined probability to drop out each of the training contextual signals during training of the contextual frontend processing model.

(4) Implementations of the disclosure may include one or more of the following optional features. In some implementations, the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with all-zeroes. In these implementations, replacing the training reference audio signal with all zeroes includes replacing the training reference audio signal with an all-zero feature of a same length and feature dimension as the corresponding training utterance. Additionally or alternatively, replacing the training contextual noise signal includes replacing the training contextual noise signal with an all-zero feature having a predetermined length and a same feature dimension as the corresponding training utterance. Additionally, in these implementations, replacing the training speaker vector includes replacing the training speaker vector with an all-zero feature with an all-zero vector. In some examples, the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with a frame-level learned representation.

(5) In some implementations, the trained contextual frontend processing model includes a primary encoder, a noise context encoder, a cross-attention encoder, and a decoder. The primary encoder receives, as input, input speech features corresponding to a target utterance and generate, as output, a main input encoding. The noise context encoder receives, as input, a contextual noise signal including noise prior to the target utterance, and generate, as output, a contextual noise encoding. The cross-attention encoder receives, as input, the main input encoding generated as output from the primary encoder and the contextual noise encoding generated as output from the noise context encoder, and generates, as output, a cross-attention embedding. The decoder decodes the cross-attention embedding into enhanced input speech features corresponding to the target utterance. In these implementations, the primary encoder is further configured to receive, as input, reference features corresponding to a reference audio signal, and generate, as output, the main input encoding by processing the input speech features stacked with the reference features. Alternatively, the primary encoder is further configured to receive, as input, a speaker embedding including voice characteristics of a target speaker that spoke the target utterance, and generate, as output, the main input encoding by combining the input speech features with the speaker embedding using feature-wise linear modulation (FiLM). Additionally or alternatively, the cross-attention encoder is further configured to receive, as input, the main input encoding modulated by a speaker embedding using feature-wise linear modulation (FiLM). Here, the speaker embedding including voice characteristics of a target speaker that spoke the target utterance, and process the main input encoding modulated by the speaker embedding and the contextual noise encoding to generate, as output, the cross-attention embedding. In some implementations, the primary encoder includes N modulated conformer blocks, the context noise encoder includes N conformer blocks and executes in parallel with the primary encoder, and the cross-attention encoder includes M modulated cross-attention conformer blocks.

(6) In some examples, the contextual frontend processing model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss. In these examples, the spectral loss may be based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask. Here, the ideal ratio mask is computed using reverberant speech and reverberant noise. Additionally, in these examples, the ASR loss is computed by receiving enhanced speech features predicted by the contextual frontend processing model for a training utterance as input, predicted outputs of the ASR encoder for the enhanced speech features, generating, using the ASR encoder configured to receive target speech features for the training utterance as input, target outputs of the ASR encoder for the target speech features, and computing the ASR loss based on the predicted outputs of the ASR encoder for the enhanced speech features and the target outputs of the ASR encoder for the target speech features.

(7) Another aspect of the disclosure provides a system for training for a generalized automatic speech recognition model for joint echo cancellation, speech enhancement, and voice separation. The system includes data processing hardware and memory hardware in communication with the data processing hardware. The memory hardware stores instructions that when executed on the data processing hardware cause the date processing hardware to perform operations including receiving a plurality of training utterances paired with corresponding training contextual signals. The training contextual signals include a training contextual noise signal including noise prior to the corresponding training utterance, a training reference audio signal, and a training speaker vector including voice characteristics of a target speaker that spoke the corresponding training utterance. The operations also include training, using a contextual signal dropout strategy, a contextual frontend processing model on the training utterances to learn how to predict enhanced speech features. Here, the contextual signal dropout strategy uses a predetermined probability to drop out each of the training contextual signals during training of the contextual frontend processing model.

(8) This aspect may include one or more of the following optional features. In some implementations, the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with all-zeroes. In these implementations, replacing the training reference audio signal with all zeroes includes replacing the training reference audio signal with an all-zero feature of a same length and feature dimension as the corresponding training utterance. Additionally or alternatively, replacing the training contextual noise signal includes replacing the training contextual noise signal with an all-zero feature having a predetermined length and a same feature dimension as the corresponding training utterance. Additionally, in these implementations, replacing the training speaker vector includes replacing the training speaker vector with an all-zero feature with an all-zero vector. In some examples, the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with a frame-level learned representation.
(9) In some implementations, the trained contextual frontend processing model includes a primary encoder, a noise context encoder, a cross-attention encoder, and a decoder. The primary encoder receives, as input, input speech features corresponding to a target utterance and generate, as output, a main input encoding. The noise context encoder receives, as input, a contextual noise signal including noise prior to the target utterance, and generate, as output, a contextual noise encoding. The cross-attention encoder receives, as input, the main input encoding generated as output from the primary encoder and the contextual noise encoding generated as output from the noise context encoder, and generates, as output, a cross-attention embedding. The decoder decodes the cross-attention embedding into enhanced input speech features corresponding to the target utterance. In these implementations, the primary encoder is further configured to receive, as input, reference features corresponding to a reference audio signal, and generate, as output, the main input encoding by processing the input speech features stacked with the reference features. Alternatively, the primary encoder is further configured to receive, as input, a speaker embedding including voice characteristics of a target speaker that spoke the target utterance, and generate, as output, the main input encoding by combining the input speech features with the speaker embedding using feature-wise linear modulation (FiLM). Additionally or alternatively, the cross-attention encoder is further configured to receive, as input, the main input encoding modulated by a speaker embedding using feature-wise linear modulation (FiLM). Here, the speaker embedding including voice characteristics of a target speaker that spoke the target utterance, and process the main input encoding modulated by the speaker embedding and the contextual noise encoding to generate, as output, the cross-attention embedding. In some implementations, the primary encoder includes N modulated conformer blocks, the context noise encoder includes N conformer blocks and executes in parallel with the primary encoder, and the cross-attention encoder includes M modulated cross-attention conformer blocks.
(10) In some examples, the contextual frontend processing model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss. In these examples, the spectral loss may be based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask. Here, the ideal ratio mask is computed using reverberant speech and reverberant noise. Additionally, in these examples, the ASR loss is computed by receiving enhanced speech features predicted by the contextual frontend processing model for a training utterance as input, predicted outputs of the ASR encoder for the enhanced speech features, generating, using the ASR encoder configured to receive target speech features for the training utterance as input, target outputs of the ASR encoder for the target speech features, and computing the ASR loss based on the predicted outputs of the ASR encoder for the enhanced speech features and the target outputs of the ASR encoder for the target speech features.
(11) The details of one or more implementations of the disclosure are set forth in the accompanying drawings and the description below. Other aspects, features, and advantages will be apparent from the description and drawings, and from the claims.

Description

DESCRIPTION OF DRAWINGS
(1) FIG. **1** is a schematic view of a system that includes a user communicating a spoken target utterance to a speech-enabled user device.
(2) FIG. **2** is a schematic view of a contextual frontend processing model of FIG. **1**.
(3) FIG. **3** is a schematic view of a modulated conformer block.
(4) FIG. **4** is a schematic view of a modulated conformer block architecture implemented by a cross-attention encoder of the contextual frontend processing model.
(5) FIG. **5** is a schematic view of an example training process for training a contextual frontend processing model.
(6) FIG. **6** is a schematic view of an example training process for jointly training a contextual frontend processing model and an automatic speech recognition model.
(7) FIG. **7** is an example flowchart of an example arrangement of operations for a method of automatic speech recognition using a contextual frontend processing model.
(8) FIG. **8** is a schematic view of an example computing device that may be used to implement the systems and methods described herein.
(9) Like reference symbols in the various drawings indicate like elements.
DETAILED DESCRIPTION
(10) Robustness of automatic speech recognition (ASR) systems has significantly improved over the years with the advent of neural network-based end-to-end models, large-scale training data, and improved strategies for augmenting training data. Nevertheless, background interference can significantly deteriorate the ability of ASR systems to accurately recognize speech directed toward the ASR system. Background interference can be broadly classified into three groups: device echo; background noise; and competing speech. While separate ASR models may be trained to handle each of these background interference groups in isolation, the difficulty in maintaining multiple task/condition-specific ASR models and switching between the models on the fly during use is not practical.
(11) Device echo may correspond to playback audio output from devices, such as smart home speakers, whereby the playback audio is recorded as echo and can affect performance of a backend speech system, such as an ASR system. Particularly, degradation of performance of the backend speech system is especially severe if the playback audio contains audible speech, e.g., a text-to-speech (TTS) response from a digital assistant. This problem is typically addressed via acoustic echo cancelation (AEC) techniques. A unique characteristic of AEC is that a reference signal corresponding to the playback audio is typically available and can be used for suppression.
(12) Background noise with non-speech characteristics is usually well handled using data augmentation strategies like multi-style training (MTR) of the ASR models. Here, a room simulator is used to add noise to the training data, which is then carefully weighted with clean data during training to get a good balance in performance between clean and noisy conditions. As a result, large scale ASR models are robust to moderate levels of non-speech noise. However, background noise can still affect performance of backend speech systems in the presence of low signal-to-noise ratio (SNR) conditions.
(13) Unlike non-speech background noise, competing speech is quite challenging for ASR models that are trained to recognize a single speaker. Training ASR models with multi-talker speech can pose problems in itself, since it is hard to disambiguate which speaker to focus on during inference. Using models that recognize multiple speakers is also sub-optimal since it is hard to know ahead of time how many users to support. Furthermore, such multi-speaker models typically have degraded performance in single-speaker settings, which is undesirable.
(14) The three aforementioned classes of background interference have typically been addressed in isolation of one another, each using separate modeling strategies. Speech separation has received a lot of attention in the recent literature using techniques like deep clustering, permutation invariant training, and using speaker embeddings. When using speaker embeddings, the target speaker of interest is assumed to be known a priori. Techniques developed for speaker separation have also been applied to remove non-speech noise, with modifications to the training data. AEC has

also been studied in isolation or together in the presence of background noise. It is well known that improving speech quality does not always improve ASR performance since the distortions introduced by non-linear processing can adversely affect ASR performance. One way to mitigate discrepancies between an enhancement frontend initially processing incoming audio and the resulting ASR performance is to jointly train the enhancement frontend together with the backend ASR model.

(15) Moreover, as the application of large scale multi-domain and multi-lingual ASR models continues to gain interest, the training data for these ASR models typically covers various acoustic and linguistic use cases (e.g., voice search and video captioning), thereby making it challenging to simultaneously address harsher noise conditions. As a result, it is often convenient to train and maintain separate frontend feature processing models capable of handling adverse conditions, without combining it with the backend ASR model. Furthermore, while various types of data for ASR models is available for training, the ASR model must also perform well when one or more of the aforementioned groups of background interference (e.g., device echo; background noise; and competing speech) are missing from training examples.

(16) Implementations herein are directed toward training a contextual frontend processing model for improving robustness of ASR by jointly implementing acoustic echo cancellation (AEC), speech enhancement, and speech separation modules into a single model. A single joint model is practical from the standpoint that it is difficult, if not impossible, to know what class of background interference to address ahead of time, particularly in a streaming ASR setting. Specifically, the contextual frontend processing model includes a contextual enhancement neural network (CENN) capable of optionally making use of three different types of side contextual inputs: a reference signal associated with playback audio; noise context; and a speaker embedding representing voice characteristics of a target speaker of interest. Implementations herein are more specifically directed toward using a contextual signal dropout strategy for training the contextual frontend processing model to improve performance of the model during inference when one or more contextual inputs are missing. As will become apparent, the reference signal associated with the playback audio is necessary for providing echo cancellation while the noise context is useful for speech enhancement. Additionally, the speaker embedding (when available) representing the voice characteristics of the target speaker is not only critical for speech separation, but is also helpful for echo cancelation and speech enhancement. For speech enhancement and separation, the noise context, i.e., a few seconds of audio before the target utterance to be recognized, carries useful information about the acoustic context. The CENN employs a respective neural network architecture configured to ingest each corresponding contextual side input to produce enhanced input speech features that may be passed to a backend speech system, such as, an ASR model that may process the enhanced input speech features to generate a speech recognition result for the target utterance. Notably, as the noise context and reference features are optional contextual side inputs, the noise context and reference features are assumed by the CENN to be respective uninformative silence signals when not available.

(17) Referring to FIG. **1**, in some implementations, a system **100** includes a user **10** communicating a spoken target utterance **12** to a speech-enabled user device **110** (also referred to as a device **110** or a user device **110**) in a speech environment. The user **10** (i.e., speaker of the utterance **12**) may speak the target utterance **12** as a query or a command to solicit a response from the device **110**. The device **110** is configured to capture sounds from one or more users **10**, **11** within the speech environment. Here, the audio sounds may refer to a spoken utterance **12** by the user **10** that functions as an audible query, a command for the device **110**, or an audible communication captured by the device **110**. Speech-enabled systems of the device **110** or associated with the device **110** may field the query for the command by answering the query and/or causing the command to be performed.

(18) Various types of background interference may interfere with the ability of a backend speech system **180** to process the target utterance **12** that specifies the query or command for the device **110**. As aforementioned, the background interference may include one or more of a device echo corresponding to playback audio **154** (also referred to as a reference audio signal **154**) output from the user device (e.g., a smart speaker) **110**, competing speech **13** such as utterances other than the target utterance **12** spoken by one or more other users **111** that are not directed toward the device **110**, and background noise with non-speech characteristics. Implementations herein employ a contextual frontend processing model **200** (also referred to as a model **200**) that executes on the device **110** and is configured to receive, as input, input speech features corresponding to the target utterance **12** and one or more contextual input features **213**, **214**, **215**, and generate, as output, enhanced input speech features **250** corresponding to the target utterance **12** by processing the input speech features **212** and the one or more contextual input features **213**, **214**, **215**. As described in greater detail below (e.g., FIG. **5**), the model **200** may be trained using a contextual signal dropout strategy to improve performance of the model **200** during inference when one or more of the contextual input features **213**, **214**, **215** is missing. A backend speech system **180** may then process the enhanced input speech features **250** to generate an output **182**. Notably, the contextual frontend processing model **200** effectively removes the presence of background interference recorded by the device **110** when the user **10** spoke the target utterance **12** such that the enhanced input speech features **250** provided to the backend speech system **180** convey the speech (i.e., target utterance **12**) that was intended for the device **110** so that the output **182** generated by the backend speech system **180** is not degraded by the background interference.

(19) In the example shown, the backend speech system **180** includes an ASR system **190** that employs an ASR model **192** to process the enhanced input speech features **250** to generate a speech recognition result (e.g., transcription) for the target utterance **12**. The ASR system **190** may further include a natural language understanding (NLU) module (not shown) that performs semantic interpretation on the transcription of the target utterance **12** to identify the query/command directed toward the device **110**. As such, the output **182** from the backend speech system **180** may include the transcription and/or instructions to fulfill the query/command identified by the NLU module.

(20) The backend speech system **180** may additionally or alternatively include a hotword detection model (not shown) configured to detect whether or not the enhanced input speech features **250** include a presence of one or more hotwords/warm words the hotword detection model is trained to detect. For instance, the hotword detection model may output a hotword detection score indicating a likelihood that the enhanced input speech features **250** corresponding to the target utterance **12** include a particular hotword/warm word. Detection of a hotword may trigger a wake-up process that causes the device **110** to wake-up from a sleep state. For instance, the device **110** may wake-up and process the hotword and/or one or more terms preceding/following the hotword.

(21) In additional examples, the background speech system **180** includes an audio or audio-video calling application (e.g., a video conferencing application). Here, the enhanced input speech features **250** corresponding to the target utterance **12** are used by the audio or audio-video calling application to filter the voice of the target speaker **10** for communications to recipients during an audio or audio-video communication session. The background speech system **180** may additionally or alternatively include a speaker identification model configured to perform speaker identification using the enhanced input speech features **250** to identify the user **10** that spoke the target utterance **12**.

(22) In the example shown, the device **110** captures a noisy audio signal **202** (also referred to audio data) of the target utterance **12** spoken by the user **10** in the presence of background interference emanating from one or more sources other than the user **10**. The device **110** may correspond to any computing device associated with the user **10** and capable of receiving noisy audio signals **202**. Some examples of user devices **110** include, but are not limited to, mobile devices (e.g., mobile phones, tablets, laptops, etc.), computers, wearable devices (e.g., smart watches), smart appliances, and internet of things (IoT) devices, smart speakers, etc. The device **110** includes data processing hardware **112** and memory hardware **114** in communication with the data processing hardware **112** and storing instructions, that when executed by the data processing hardware **112**, cause the data processing hardware **112** to perform one or more operations. The contextual frontend processing model **200** may execute on the data processing hardware **112**. In some examples, the backend speech system **180** executes on the data processing hardware **112**.

(23) In some examples, the device **110** includes one or more applications (i.e., software applications) where each application may utilize enhanced input speech features **250** generated by the contextual frontend processing model **200** to perform various functions within the application. For instance, the device **110** includes an assistant application configured to communicate synthesized playback audio **154** to the user **10** to assist the user **10** with various tasks.

(24) The device **110** further includes an audio subsystem with an audio capturing device (e.g., a microphone) **116** for capturing and converting

spoken utterances **12** within the speech environment into electrical signals and a speech output device (e.g., a speaker) **118** for communicating an audible audio signal (e.g., a synthesized playback signal **154** from the device **110**). While the device **110** implements a single audio capturing device **116** in the example shown, the device **110** may implement an array of audio capturing devices **116** without departing from the scope of the present disclosure, whereby one or more audio capturing devices **116** in the array may not physically reside on the device **110**, but be in communication with the audio subsystem (e.g., peripherals of the device **110**). For example, the device **110** may correspond to a vehicle infotainment system that leverages an array of microphones positioned throughout the vehicle.

(25) In some examples, the device **110** is configured to communicate with a remote system **130** via a network (not shown). The remote system **130** may include remote resources **132**, such as remote data processing hardware **134** (e.g., remote servers or CPUs) and/or remote memory hardware **136** (e.g., remote databases or other storage hardware). The device **110** may utilize the remote resources **132** to perform various functionality related to speech processing and/or synthesized playback communication. The contextual frontend processing model **200** and the backend speech system **180** may reside on the device **110** (referred to as on-device systems) or reside remotely (e.g., reside on the remote system **130**), but in communication with the device **110**. In some examples, one or more backend speech systems **180** reside locally or on-device while one or more other backend speech systems **180** reside remotely. In other words, one or more backend speech systems **180** leveraging the enhanced input speech features **250** output from the contextual frontend processing model **200** may be local or remote in any combination. For instance, when a system **180** is rather large in size or processing requirements, the system **180** may reside in the remote system **130**. Yet when the device **110** may support the size or the processing requirements of one or more systems **180**, the one or more systems **180** may reside on the device **110** using the data processing hardware **112** and/or the memory hardware **114**. Optionally, the one or more of the systems **180** may reside on both locally/on-device and remotely. For instance, a backend speech system **180** may default to execute on the remote system **130** when a connection between the device **110** and remote system **130** is available, but when the connection is lost or unavailable, the system **180** instead executes locally on the device **110**.

(26) In some implementations, the device **110** or a system associated with the device **110** identifies text that the device **110** will communicate to the user **10** as a response to a query spoken by the user **10**. The device **110** may then use a text-to-speech (TTS) system to convert the text into corresponding synthesized playback audio **154** for the device **110** to communicate to the user **10** (e.g., audibly communicate to the user **10**) as the response to the query. Once generated, the TTS system communicates the synthesized playback audio **154** to the device **110** to allow the device **110** to output the synthesized playback audio **154**. For instance, the device **110** outputs the synthesized playback audio **154** of "today is sunny" at a speaker **118** of the device **110** responsive to the user **10** providing a spoken query for today's weather forecast.

(27) With continued reference to FIG. **1**, when the device **110** outputs the synthesized playback audio **154**, the synthesized playback audio **154** generates an echo **156** captured by the audio capturing device **116**. The synthesized playback audio **154** corresponds to a reference audio signal. While synthesized playback audio **154** depicts a reference audio signal in the example of FIG. **1**, the reference audio signal may include other types of playback audio **154** including media content output from the speaker **118** or a communication from a remote user the user **10** is conversing with (e.g., voice over IP call or video conferencing call) through the device **110**. Unfortunately, in addition to the echo **156**, the audio capturing device **116** may also be simultaneously capturing the target utterance **12** spoken by the user **10** that includes a follow-up query inquiring more about the weather, by stating "what about tomorrow?" For example, FIG. **1** depicts that, as the device **110** outputs the synthesized playback audio **154**, the user **10** inquires more about the weather, in a spoken utterance **12** to the device **110**, by stating "what about tomorrow?" Here, the spoken utterance **12** and the echo **156** are both captured at the audio capturing device **116** simultaneously to form the noisy audio signal **202**. In other words, the audio signal **202** includes an overlapped audio signal where some portion of the target utterance **12** spoken by the user **10** overlaps with some portion of the reference audio signal (e.g., synthesized playback audio) **154** output from the speaker **118** of the device **110**. In addition to the synthesized playback audio **154**, competing speech **13** spoken by another user **11** in the environment may also be captured by the audio capturing device **116** and contribute to background interference that overlaps with the target utterance **12**.

(28) In FIG. **1**, the backend speech system **180** may have issues processing the target utterance **12** corresponding to the follow-up weather query "what about tomorrow?" in the noisy audio signal **202** due to the presence of the background interference attributed to at least one of the playback audio **154**, competing speech **13**, or non-speech background noise interfering with target utterance **12**. The contextual frontend processing model **200** is employed to improve robustness of the backend speech system **180** by jointly implementing acoustic echo cancellation (AEC), speech enhancement, and speech separation models/modules into a single model.

(29) In order to perform acoustic echo cancellation (AEC), the single model **200** uses the reference signal **154** that is being played back by the device as an input to the model **200**. It is assumed that the reference signal **154** is temporally aligned with the target utterance **12**, and is of the same length. In some examples, a feature extractor (not shown) extracts reference features **214** corresponding to the reference audio signal **154**. The reference features **214** may include log Mel-filterbank energy (LFBE) features of the reference audio signal **154**. Similarly, the feature extractor may extract input speech features **212** corresponding to the target utterance **12**. The input speech features **212** may include LFBE features. As described in greater detail below, the input speech features **212** may be stacked with the reference features **214** and provided as input to a primary encoder **210** (FIG. **2**) of the single model **200** to perform AEC. When there is no reference audio signal **154** being played by the device, an all-zero reference signal may be used such that only the input speech features **212** are received as input to the primary encoder **210**.

(30) The single model **200** may additionally perform speech enhancement in parallel with AEC by applying noise context modeling where the single model **200** processes a contextual noise signal **213** associated with a predetermined duration of noise segments captured by the audio capturing device **116** prior to the target utterance **12** spoken by the user **10**. In some examples, the predetermined duration includes six (6) seconds of noise segments. As such, the contextual noise signal **213** provides noise context. In some examples, the contextual noise signal **213** includes LFBE features of the noise context signal for use as contextual information.

(31) Optionally, the single model **200** may additionally perform target speaker modeling for speech separation jointly with AEC and speech enhancement. Here, a speaker embedding **215** is received as input by the single model **200**. The speaker embedding **215** may include voice characteristics of the target speaker **10** that spoke the target utterance **12**. The speaker embedding **215** may include a d-vector. In some examples, the speaker embedding **215** is computed using a text-independent speaker identification (TI-SID) model trained with a generalized end-to-end extended-set softmax loss. The TI-SID may include three long short-term memory (LSTM) layers with **768** nodes and a projection size of 256. The output of the final frame of the last LSTM layer is then linearly transformed to the final 256-dimension d-vector.

(32) For training and evaluations, each target utterance may be paired with a separate "enrollment" utterance from the same speaker. The enrollment utterance may be randomly selected from a pool of available utterances of the target speaker. The d-vectors are then computed on the enrollment utterance. For most real applications, the enrollment utterances are usually obtained via a separate offline process.

(33) FIG. **2** shows the contextual frontend processing model **200** of FIG. **1**. The contextual frontend processing model **200** uses a modified version of a conformer neural network architecture that combines convolution and self-attention to model short-range and long-range interactions. The model **200** includes a primary encoder **210**, a noise context encoder **220**, a cross-attention encoder **400**, and a decoder **240**. The primary encoder **210** may include N modulated conformer blocks. The noise context encoder **220** may include N conformer blocks. The cross-attention encoder **230** may include M modulated cross-attention conformer blocks. The primary and noise context encoders **210**, **220** may execute in parallel. As used herein, each conformer block may use local, causal self-attention to allow for streaming capabilities.

(34) The primary encoder **210** may be configured to receive, as input, input speech features **212** corresponding to the target utterance, and generate, as output, a main input encoding **218**. When the reference audio signal **154** is available, the primary encoder **210** is configured to receive the input speech features **212** stacked with reference features **214** corresponding to the reference audio signal as input and generate the main input encoding by processing the input speech features **212** stacked with the reference features **214**. The input speech features and the reference features may each

include a respective sequence of LFBE features.

(35) The primary encoder **210** may be further configured to receive, as input, the speaker embedding **215** (i.e., when available) including the voice characteristics of the target speaker (i.e., the user) **10** that spoke the target utterance **12**, and generate, as output, the main input encoding **218** by combining the input speech features **212** (or the input speech features stacked with the reference features **214**) using a feature-wise linear modulation (FiLM) layer **310** (FIG. **3**). FIG. **3** provides an example modulated conformer block **320** employed by the primary encoder **210**. Here, before each conformer block **320** at the primary encoder **210**, the speaker embedding **215** (e.g., d-vector) is combined with the input speech features **212** (or stack of input speech and reference features **214**) using the FiLM layer **310** to generate an output **312**. FiLM permits the primary encoder **210** to adjust its encoding based on the speaker embedding **215** of the target speaker **10**. A residual connection **314** is added after the FiLM layer **310** to combine the input speech features **212** (or the input speech features **212** stacked with the reference features **214**) with the output **312** of the FiLM layer **310** to generate modulated input features **316** as input for the conformer block **320** in order to ensure that the architecture can perform well when the speaker embedding **215** is absent. Mathematically, the modulated conformer block **320** transforms input features x, using modulation features m, to produce output features y, as follows:

(36) $$\tilde{x} = x + r(m) \odot x + h(m)x' = \tilde{x} + \tfrac{1}{2}FFN(\tilde{x})x'' = x' + \mathrm{Conv}(x')x''' = x'' + \mathrm{MHSA}(x)y = \mathrm{LayerNorm}(x''' + \tfrac{1}{2}\mathrm{FFN}(x''')). \quad (1)$$

(37) Here, h (.Math.) and r (.Math.) are affine transformations. FFN, Cony, and MHSA stand for feed-forward module, convolution module, and multi-headed self-attention module, respectively. Eq. 1 shows the feature-wise linear modulation (FiLM) layer **310**, with the residual connection.

(38) Referring back to FIG. **2**, the noise context encoder **220** is configured to receive, as input, a contextual noise signal **213** that includes the noise prior to the target utterance, and generate, as output, a contextual noise encoding **222**. The contextual noise signal **213** may include LFBE features of the contextual noise signal. The noise context encoder **220**, unlike the primary and cross-attention encoders **210**, **400**, includes standard conformer blocks without modulation by the speaker embedding **215**. The noise context encoder **220** does not modulate the contextual noise signal **213** with the speaker embedding **215** since the contextual noise signal **213** is associated with acoustic noise context prior to the target utterance **12** is spoken, and thus, is assumed to contain information that should be passed forward to the cross-attention encoder **400** to aid with noise suppression.

(39) With continued reference to FIG. **2**, the cross-attention encoder **400** may be configured to receive, as input, the main input encoding **218** generated as output from the primary encoder **210** and the contextual noise encoding **222** generated as output from the noise context encoder **220**, and generate, as output, a cross-attention embedding **480**. Thereafter, the decoder **240** is configured to decode the cross-attention embedding **480** into the enhanced input speech features **250** corresponding to the target utterance **12**. The contextual noise encoding **222** may correspond to an auxiliary input. The decoder **240** may include a simple projection decoder having a single layer, frame-wise fully connected network with sigmoid activation.

(40) As shown in FIG. **4**, the cross-attention encoder **400** may employ a respective set of M modulated conformer blocks that each receive, as input, the main input encoding **218** modulated by the speaker embedding **215** using FiLM as described in FIG. **3** and the contextual noise encoding **222** output from the noise context encoder **220**. The cross-attention encoder **400** first independently processes the modulated input **218** and the auxiliary input **222** using half feed-forward nets **402**, first residual connections **404**, convolutional blocks **406**, and second residual connections **408**. Specifically, the modulated input **218** is processed by a half feed-forward net **402***a*, which generates an output **403***a*. Next, a first residual connection **404***a* combines the modulated input **218** with the output **403***a* of the half-feedforward net **402***a* to generate modulated input features **405***a*. The modulated input features **405***a* are input to a convolution block **406***a*, which generates a convolutional output **407***a*. A second residual connection **408***a* combines the convolutional output **407***a* of the convolution block **406***a* with the modulated input features **405***a* to generate an output including a query vector **409***a*.

(41) Likewise, the auxiliary input **222** is processed by a half feed-forward net **402***b*, which generates an output **403***b*. Next, a first residual connection **404***b* combines the auxiliary input **222** with the output **403***b* of the half-feedforward net **402***b* to generate modulated input features **405***b*. The modulated input features **405***b* are input to a convolution block **406***b*, which generates a convolutional output **407***b*. A second residual connection **408***b* combines the convolutional output **407***b* of the convolution block **406***b* with the modulated input features **405***b* to generate an output including a first key vector **409***b* and a first value vector **409***c*.

(42) Subsequently, a multi-head cross attention (MHCA) module **410** receives, as input, the query vector **409***a*, the first key vector **409***b*, and the first value vector **409***c*, and summarizes these vectors **409***a*—c to generate a noise summary **412**. Intuitively, the role of the MHCA module **410** is to summarize noise context separately for each input frame that is to be enhanced. The noise summary **412** output by the MHCA module **410** is then merged with the query vector **409***a* using a FiLM layer **420**, which generates an FiLM output **422**.

(43) A multi-head self-attention (MHSA) layer **430** receives the FiLM output **422** as input and merges the FiLM output **422** with the query vector **409***a* to generate an attention output **432**. A third residual connection **434** receives the query vector **409***a* and the attention output **432** and combines the query vector **409***a* and the attention output **432** to generate a residual output **436**. A feed forward module **440** then receives the residual output **436** of the third residual connection **434** as input and generates a features output **442**. Next, a fourth residual connection **444** combines the features output **422** with the residual output **436** of the third residual output **434** to generate merged input features **446**. The merged input features **446** are then processed as input by a layernorm **450**, which to a convolution block **406***b*, which generates a cross-attention embedding **480**.

(44) Mathematically, if x, m, and n are the encoded input, d-vector and the encoded noise context from the previous layer, the cross attention encoder **400** performs the following:

(45)

$$\tilde{x} = x + r(m) \odot x + h(m)\tilde{x} = \tilde{x} + \tfrac{1}{2}\mathrm{FFN}(\tilde{x}), \tilde{n} = n + \tfrac{1}{2}\mathrm{FFN}(n)x' = \tilde{x} + Conv(\tilde{x}), n' = \tilde{n} + \mathrm{Conv}(\tilde{n})x'' = \mathrm{MHCA}(x', n')x''' = x' + x' \odot r(x'') + h(x'')x'''' = x'''$$

(46) The cross attention encoder **400** generates, as an output, the cross-attention embedding **480**, which is passed on to the next layer of the M modulated conformer blocks, along with the d-vector m, and the encoded noise context n. Thus, inputs are modulated by each of the M conformer blocks by both the speaker embedding **215** associated with the target speaker and the noise context encoding **222**.

(47) FIG. **5** shows an example training process **500** for training the contextual frontend processing model **200** to generate enhanced input speech features **250** when one or more of the contextual input features **213**, **214**, **215** are not present. The training process **500** may execute on the remote system **130** of FIG. **1**. As shown, the training process obtains one or more training data sets **520** stored in a data store **510** and trains the contextual frontend processing model **200** on the training data sets **520**. The data store **510** may reside on the memory hardware **136** of the remote system **130**. Each training data set **520** includes a plurality of training examples, **530**, **530***a*—n, where each training example **530** may include a training utterance **532** paired with corresponding training contextual signals **534**, **534***a*—c. Specifically, the training contextual signals **534** include a training contextual noise signal **534***a* including noise prior to the corresponding training utterance **532**, a training reference audio signal **534***b*, and a training speaker vector **534***c* including voice characteristics of a target speaker that spoke the corresponding training utterance **532**.

(48) As discussed above with respect to FIG. **1**, during inference, the contextual frontend processing model **200** may not receive all of the contextual input features **213**, **214**, **215** at the same time. Training the contextual frontend processing model **200** with one or more missing training contextual signals **534** encourages the contextual frontend processing model **200** to utilize alternates in the contextual input features **213**, **214**, **215** rather than overly rely on the most relevant of the contextual input features **213**, **214**, **215**. As a result, the contextual frontend processing model **200** can accurately predict enhanced input speech features **250** when one or more of the contextual input features **213**, **214**, **215** is not present. In order to keep the contextual frontend processing model **200** static, any missing training contextual signals **534** still need to be input to the contextual frontend processing model **200** in some manner.

(49) The training process **500** may also utilize a signal dropout model **550**. The signal dropout model **550** receives the training contextual signals **534** as input from the data store **510** and, using a contextual signal dropout strategy, drops out one or more of the training contextual signals **534** prior to

training the contextual frontend processing model **200**. The contextual signal dropout strategy of the signal dropout model **550** may include a predetermined probability (e.g., 50%, 20%, etc.,) to drop out each of the training contextual signals **534**, where the same predetermined probability is used for each of the training contextual signals **534**. In other words, in a given training example **530**, the signal dropout model **550** may, using the contextual signal dropout strategy, drop out the training contextual noise signal **534***a* at a predetermined probability of 50%, the training reference audio signal **534***b* at a predetermined probability of 50%, and the training speaker vector **534***c* at a predetermined probability of 50%. Likewise, in a given training example, the signal dropout model **550** may, using the contextual signal dropout strategy, drop out the training contextual noise signal **534***a* at a predetermined probability of 20%, the training reference audio signal **534***b* at a predetermined probability of 20%, and the training speaker vector **534***c* at a predetermined probability of 20%.

(50) In addition to the signal dropout strategy, the signal dropout model **550** may trim the length of the training contextual noise signal **534***a* to include noise prior to the corresponding training utterance **532** with a uniformly distributed length of zero to six (0-6) seconds. In other words, the signal dropout model **550** implements the signal dropout strategy and trims the training contextual noise signal **534***a* concurrently. For example, for a given training example **530**, even if the signal dropout model **550** does not drop out the training contextual noise signal **534***a*, the signal dropout model **550** may still trim the length of the training contextual noise signal **534***a*.

(51) In some implementations, the signal dropout model **550** uses the signal dropout strategy to, based on the predetermined probability, drop out each training contextual signal **534** by replacing the corresponding training contextual signal **534** with all-zeroes. In these implementations, the signal dropout model **550** may replace the training contextual noise signal **534***a* with an all-zero feature having a predetermined length and a same feature dimension as the corresponding training utterance **532**. For example, the signal dropout strategy includes creating an all-zero feature with a length of six (6) seconds, and the same dimension as the LFBE features. Similarly, the signal dropout model **550**, using the signal dropout strategy, may replace the training reference audio signal **534***b* with an all-zero feature of a same length and feature dimension as the corresponding training utterance **532**. Here, the feature dimension of the all-zero training reference audio signal **534***b* corresponds to the LFBE features of the training reference audio signal **534***b* if the signal dropout strategy had not dropped out the training reference audio signal **534***b*. Likewise, the signal dropout model **550**, using the signal dropout strategy, may replace the training speaker vector **534***c* with an all-zero feature with an all-zero vector. Here, the training speaker vector **534***c* is replaced with a 256-dimensional all-zero vector. In other implementations, the signal dropout model **550** uses the signal dropout strategy to, based on the predetermined probability, drop out each training contextual signal **534** by replacing the corresponding training contextual signal **534** with a frame-level learned representation.

(52) In the example shown in FIG. **5**, the signal dropout model **550** receives the training contextual signals **534***a*—c as input and, using the predetermined probability of the contextual signal dropout strategy, drops out the training reference audio signal **534***b* by replacing the training reference audio signal **534***b* with an all-zero feature of a same length and feature dimension as the corresponding training utterance **532**. In other words, at this time-step, the contextual frontend processing model **200** is only trained on the training contextual signals **534***a*, **534***c*, which approximates a condition the model **200** may encounter during inference where the contextual input features **213**, **214**, **215** only include a contextual noise signal **213** and a speaker embedding **215**.

(53) After the signal dropout model **550** drops out the training reference audio signal **534***b*, the training utterance **532** and the training contextual signals **534** including the training reference audio signal **534***b* replaced with the all-zero feature and dimension to simulate the training reference audio signal **534***b* as missing are provided to train the contextual frontend processing model **200**. The contextual frontend processing model **200** receives, as input, the training utterance **532** and the training contextual signals **534** simulating the training reference audio signal **534***b* as missing and generates an output prediction y.sub.r. The output prediction y.sub.r includes enhanced input speech features **250**, which is tested for its accuracy. At each time-step during the training process **500**, the contextual frontend processing model **200** is additionally trained using the output prediction for the previous time-step y.sub.r-1.

(54) FIG. **6** shows an example training process **600** for computing ASR loss **640** when the contextual frontend processing model **200** is trained jointly with the ASR model **192**. Here, only an encoder **620** of the ASR model **192** is used for computing the loss. The ASR loss **640** is computed as the **12** distance between the outputs of the ASR encoder **620** for target features **540** of the training utterance **532** and the enhanced input speech features **250**. The ASR encoder **620** is not updated during the training process **600**. In detail, the training process **600** computes the ASR loss **640** by generating, using the ASR encoder **620** of the ASR model **192** configured to receive enhanced input speech features **250** predicted by the contextual frontend processing model **200** for a training utterance **532** as input, predicted outputs **622** of the ASR encoder **620** for the enhanced input speech features **250**, and generating, using the ASR encoder **620** configured to receive target speech features **540** for the training utterance **532** as input, target outputs **624** of the ASR encoder **620** for the target speech features **540**. The predicted outputs **622** for the enhanced input speech features **250** and the target outputs **624** for the target speech features **540** may each include respective sequences of LFBE features. Thereafter, the training process **600**, via a loss module **630**, computes the ASR loss **640** based on the predicted outputs **622** of the ASR encoder **620** for the enhanced input speech features **250** and the target outputs **624** of the ASR encoder **620** for the target speech features **540**. The goal of using the ASR loss **640** is to make enhancements to the contextual frontend processing model **200** to be more attuned to the ASR model **192**, which is critical for getting the best performance out of the contextual frontend processing model **200**. By keeping the parameters of the ASR model **192** fixed, the ASR model **192** is decoupled from the contextual frontend processing model **200**, thereby allowing each to be trained and deployed independent of each other.

(55) In some implementations, the contextual frontend processing model **200** is trained jointly with the ASR model **192** of the backend automatic speech recognition system **180** using a spectral loss and the ASR loss **640**. The training target **540** for training the contextual frontend processing model **200** uses ideal ratio mask (IRM). IRMs are computed using reverberant speech and reverbant noise based on an assumption that speech and noise are uncorrelated in Mel spectral space as follows.

(56) $$M(t,c) = \frac{X(t,c)}{X(t,c)+N(t,c)} \quad (3)$$

Here, X and N are the reverberant speech and reverberant noise Mel spectrograms, respectively. t and c, represent time and Mel frequency bin indices. The choice to estimate IRMs is based on the targets being bounded between [0, 1], simplifying the estimation process. Moreover, the ASR model used for evaluation may be trained on real and simulated reverberant data, resulting in a trained ASR model that is relatively robust to reveberant speech. Therefore, IRMs derived using reverberant speech as the target still provide substantial gains in performance. The spectral loss during training are computed based L1 and L2 losses between the IRM and estimated IRM, M as follows.

(57)

$$\mathcal{L} = \sum_{t,c} \left| M(t,c) - \hat{M}(t,c) \right| + \left(M(t,c) - \hat{M}(t,c)\right)^2 \quad \text{Where} \quad L1 = \left| M(t,c) - \hat{M}(t,c) \right|, \text{ and } L2 = \left(M(t,c) - \hat{M}(t,c)\right)^2. \quad (4)$$

(58) During inference, the estimated IRM is scaled and floored to reduce speech distortion at the expense of reduced noise suppression. This is especially important, since the ASR model **192** is sensitive to speech distortions and non-linear frontend processing, which is one of the main challenges in improving performance of robust ASR models using enhancement frontends. The enhanced feature is derived as follows.

{circumflex over (X)}(*t*,*c*)=*Y*(*t*,*c*)⊙max(*{circumflex over (M)}*(*t*,*c*),β.sup.α (5)

Here, Y is the noisy Mel spectrogram, {circumflex over (X)} is an estimate of clean Mel spectrogram, α and β are exponential mask scalars, and mask floor. In some examples, α is set 0.5, and β is set to 0.01. The enhanced features may be log-compressed, i.e. log({circumflex over (X)}), and passed to the ASR model **192** for evaluation.

(59) FIG. **7** includes a flowchart of an example arrangement of operations for a method **700** of training a generalized automatic speech recognition model using a contextual frontend processing model **200**. At operation **702**, the method **700** includes receiving a plurality of training utterances **532**

paired with corresponding training contextual signals **534**, **534***a*—c. The training contextual signals **534** include a training contextual noise signal **534***a* including noise prior to the corresponding training utterance **532**, a training reference audio signal **534***b*, and a training speaker vector **534***c* including voice characteristics of a target speaker that spoke the corresponding training utterance **532**. The method **700** also includes, at operation **704**, training, using a contextual signal dropout strategy, the contextual frontend processing model **200** on the training utterances **532** to learn how to predict enhanced input speech features **250**. Here, the contextual signal dropout strategy uses a predetermined probability to drop out each of the training contextual signals **534** during training of the contextual frontend processing model **200** to simulate one or more of the training contextual signals **534** as being missing to teach the model **200** to learn how to robustly generate enhanced speech features **250** when any of the corresponding contextual input features are missing during inference.

(60) FIG. **8** is schematic view of an example computing device **800** that may be used to implement the systems and methods described in this document. The computing device **800** is intended to represent various forms of digital computers, such as laptops, desktops, workstations, personal digital assistants, servers, blade servers, mainframes, and other appropriate computers. The components shown here, their connections and relationships, and their functions, are meant to be exemplary only, and are not meant to limit implementations of the disclosures described and/or claimed in this document.

(61) The computing device **800** includes a processor **810**, memory **820**, a storage device **830**, a high-speed interface/controller **840** connecting to the memory **820** and high-speed expansion ports **850**, and a low speed interface/controller **860** connecting to a low speed bus **870** and a storage device **830**. Each of the components **810**, **820**, **830**, **840**, **850**, and **860**, are interconnected using various busses, and may be mounted on a common motherboard or in other manners as appropriate. The processor **810** (e.g., data processing hardware **112**, **134** of FIG. **1**) can process instructions for execution within the computing device **800**, including instructions stored in the memory **820** or on the storage device **830** to display graphical information for a graphical user interface (GUI) on an external input/output device, such as display **880** coupled to high speed interface **840**. In other implementations, multiple processors and/or multiple buses may be used, as appropriate, along with multiple memories and types of memory. Also, multiple computing devices **800** may be connected, with each device providing portions of the necessary operations (e.g., as a server bank, a group of blade servers, or a multi-processor system).

(62) The memory **820** (e.g., memory hardware **114**, **136** of FIG. **1**) stores information non-transitorily within the computing device **800**. The memory **820** may be a computer-readable medium, a volatile memory unit(s), or non-volatile memory unit(s). The non-transitory memory **820** may be physical devices used to store programs (e.g., sequences of instructions) or data (e.g., program state information) on a temporary or permanent basis for use by the computing device **800**. Examples of non-volatile memory include, but are not limited to, flash memory and read-only memory (ROM)/programmable read-only memory (PROM)/erasable programmable read-only memory (EPROM)/electronically erasable programmable read-only memory (EEPROM) (e.g., typically used for firmware, such as boot programs). Examples of volatile memory include, but are not limited to, random access memory (RAM), dynamic random access memory (DRAM), static random access memory (SRAM), phase change memory (PCM) as well as disks or tapes.

(63) The storage device **830** is capable of providing mass storage for the computing device **800**. In some implementations, the storage device **830** is a computer-readable medium. In various different implementations, the storage device **830** may be a floppy disk device, a hard disk device, an optical disk device, or a tape device, a flash memory or other similar solid state memory device, or an array of devices, including devices in a storage area network or other configurations. In additional implementations, a computer program product is tangibly embodied in an information carrier. The computer program product contains instructions that, when executed, perform one or more methods, such as those described above. The information carrier is a computer- or machine-readable medium, such as the memory **820**, the storage device **830**, or memory on processor **810**.

(64) The high speed controller **840** manages bandwidth-intensive operations for the computing device **800**, while the low speed controller **860** manages lower bandwidth-intensive operations. Such allocation of duties is exemplary only. In some implementations, the high-speed controller **840** is coupled to the memory **820**, the display **880** (e.g., through a graphics processor or accelerator), and to the high-speed expansion ports **850**, which may accept various expansion cards (not shown). In some implementations, the low-speed controller **860** is coupled to the storage device **830** and a low-speed expansion port **890**. The low-speed expansion port **890**, which may include various communication ports (e.g., USB, Bluetooth, Ethernet, wireless Ethernet), may be coupled to one or more input/output devices, such as a keyboard, a pointing device, a scanner, or a networking device such as a switch or router, e.g., through a network adapter.

(65) The computing device **800** may be implemented in a number of different forms, as shown in the figure. For example, it may be implemented as a standard server **800***a* or multiple times in a group of such servers **800***a*, as a laptop computer **800***b*, or as part of a rack server system **800***c*.

(66) Various implementations of the systems and techniques described herein can be realized in digital electronic and/or optical circuitry, integrated circuitry, specially designed ASICs (application specific integrated circuits), computer hardware, firmware, software, and/or combinations thereof. These various implementations can include implementation in one or more computer programs that are executable and/or interpretable on a programmable system including at least one programmable processor, which may be special or general purpose, coupled to receive data and instructions from, and to transmit data and instructions to, a storage system, at least one input device, and at least one output device.

(67) A software application (i.e., a software resource) may refer to computer software that causes a computing device to perform a task. In some examples, a software application may be referred to as an "application," an "app," or a "program." Example applications include, but are not limited to, system diagnostic applications, system management applications, system maintenance applications, word processing applications, spreadsheet applications, messaging applications, media streaming applications, social networking applications, and gaming applications.

(68) The non-transitory memory may be physical devices used to store programs (e.g., sequences of instructions) or data (e.g., program state information) on a temporary or permanent basis for use by a computing device. The non-transitory memory may be volatile and/or non-volatile addressable semiconductor memory. Examples of non-volatile memory include, but are not limited to, flash memory and read-only memory (ROM)/programmable read-only memory (PROM)/erasable programmable read-only memory (EPROM)/electronically erasable programmable read-only memory (EEPROM) (e.g., typically used for firmware, such as boot programs). Examples of volatile memory include, but are not limited to, random access memory (RAM), dynamic random access memory (DRAM), static random access memory (SRAM), phase change memory (PCM) as well as disks or tapes.

(69) These computer programs (also known as programs, software, software applications or code) include machine instructions for a programmable processor, and can be implemented in a high-level procedural and/or object-oriented programming language, and/or in assembly/machine language. As used herein, the terms "machine-readable medium" and "computer-readable medium" refer to any computer program product, non-transitory computer readable medium, apparatus and/or device (e.g., magnetic discs, optical disks, memory, Programmable Logic Devices (PLDs)) used to provide machine instructions and/or data to a programmable processor, including a machine-readable medium that receives machine instructions as a machine-readable signal. The term "machine-readable signal" refers to any signal used to provide machine instructions and/or data to a programmable processor.

(70) The processes and logic flows described in this specification can be performed by one or more programmable processors, also referred to as data processing hardware, executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application specific integrated circuit). Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data

from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and CD ROM and DVD-ROM disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

(71) To provide for interaction with a user, one or more aspects of the disclosure can be implemented on a computer having a display device, e.g., a CRT (cathode ray tube), LCD (liquid crystal display) monitor, or touch screen for displaying information to the user and optionally a keyboard and a pointing device, e.g., a mouse or a trackball, by which the user can provide input to the computer. Other kinds of devices can be used to provide interaction with a user as well; for example, feedback provided to the user can be any form of sensory feedback, e.g., visual feedback, auditory feedback, or tactile feedback; and input from the user can be received in any form, including acoustic, speech, or tactile input. In addition, a computer can interact with a user by sending documents to and receiving documents from a device that is used by the user; for example, by sending web pages to a web browser on a user's client device in response to requests received from the web browser.

(72) A number of implementations have been described. Nevertheless, it will be understood that various modifications may be made without departing from the spirit and scope of the disclosure. Accordingly, other implementations are within the scope of the following claims.

Claims

1. A computer-implemented method that when executed on data processing hardware causes the data processing hardware to perform operations comprising: receiving a plurality of training utterances paired with corresponding training contextual signals, the training contextual signals comprising: a training contextual noise signal comprising noise prior to the corresponding training utterance; a training reference audio signal; and a training speaker vector comprising voice characteristics of a target speaker that spoke the corresponding training utterance; and training, using a contextual signal dropout strategy, a contextual frontend processing model on the training utterances to learn how to predict enhanced speech features by, for each training utterance of the plurality of training utterances: generating, using a speech encoder configured to receive enhanced input speech features predicted by the contextual frontend processing model for the training utterance as input using the contextual signal dropout strategy, predicted outputs of the speech encoder for the enhanced input speech features; generating, using the speech encoder configured to receive target speech features for the training utterance as input, target outputs of the speech encoder for the target speech features; and computing a loss based on the predicted outputs of the speech encoder for the enhanced speech features and the target outputs of the speech encoder for the target speech features, wherein the contextual signal dropout strategy uses a predetermined probability to drop out each of the training contextual signals during training of the contextual frontend processing model.

2. The computer-implemented method of claim 1, wherein the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with all-zeroes.

3. The computer-implemented method of claim 2, wherein replacing the training reference audio signal with all zeroes comprises replacing the training reference audio signal with an all-zero feature of a same length and feature dimension as the corresponding training utterance.

4. The computer-implemented method of claim 2, wherein replacing the training contextual noise signal comprises replacing the training contextual noise signal with an all-zero feature having a predetermined length and a same feature dimension as the corresponding training utterance.

5. The computer-implemented method of claim 2, wherein replacing the training speaker vector comprises replacing the training speaker vector with an all-zero feature with an all-zero vector.

6. The computer-implemented method of claim 1, wherein the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with a frame-level learned representation.

7. The computer-implemented method of claim 1, wherein the trained contextual frontend processing model comprises: a primary encoder configured to: receive, as input, input speech features corresponding to a target utterance; and generate, as output, a main input encoding; a noise context encoder configured to: receive, as input, a contextual noise signal comprising noise prior to the target utterance; and generate, as output, a contextual noise encoding; and a cross-attention encoder configured to: receive, as input, the main input encoding generated as output from the primary encoder and the contextual noise encoding generated as output from the noise context encoder; and generate, as output, a cross-attention embedding; and a decoder configured to decode the cross-attention embedding into enhanced speech features corresponding to the target utterance.

8. The computer-implemented method of claim 7, wherein the primary encoder is further configured to: receive, as input, reference features corresponding to a reference audio signal; and generate, as output, the main input encoding by processing the input speech features stacked with the reference features.

9. The computer-implemented method of claim 7, wherein the primary encoder is further configured to: receive, as input, a speaker embedding comprising voice characteristics of a target speaker that spoke the target utterance; and generate, as output, the main input encoding by combining the input speech features with the speaker embedding using feature-wise linear modulation (FILM).

10. The computer-implemented method of claim 7, wherein the cross-attention encoder is further configured to: receive, as input, the main input encoding modulated by a speaker embedding using feature-wise linear modulation (FILM), the speaker embedding comprising voice characteristics of a target speaker that spoke the target utterance; and process the main input encoding modulated by the speaker embedding and the contextual noise encoding to generate, as output, the cross-attention embedding.

11. The computer-implemented method of claim 7, wherein: the primary encoder comprises N modulated conformer blocks; the noise context encoder comprises N conformer blocks and executes in parallel with the primary encoder; and the cross-attention encoder comprises M modulated cross-attention conformer blocks.

12. The computer-implemented method of claim 1, wherein the contextual frontend processing model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss.

13. The computer-implemented method of claim 12, wherein the spectral loss is based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask, the ideal ratio mask computed using reverberant speech and reverberant noise.

14. A system comprising: data processing hardware; and memory hardware in communication with the data processing hardware and storing instructions that when executed on the data processing hardware causes the data processing hardware to perform operations comprising: receiving a plurality of training utterances paired with corresponding training contextual signals, the training contextual signals comprising: a training contextual noise signal comprising noise prior to the corresponding training utterance; a training reference audio signal; and a training speaker vector comprising voice characteristics of a target speaker that spoke the corresponding training utterance; and training, using a contextual signal dropout strategy, a contextual frontend processing model on the training utterances to learn how to predict enhanced speech features by, for each training utterance of the plurality of training utterances: generating, using a speech encoder configured to receive enhanced input speech features predicted by the contextual frontend processing model for the training utterance as input using the contextual signal dropout strategy, predicted outputs of the speech encoder for the enhanced input speech features; generating, using the speech encoder configured to receive target speech features for the training utterance as input, target outputs of the speech encoder for the target speech features; and computing a loss based on the predicted outputs of the speech encoder for the enhanced speech features and the target outputs of the speech encoder for the target speech features, wherein the contextual signal dropout strategy uses a predetermined probability to drop out each of the training contextual signals during training of the contextual frontend processing model.

15. The system of claim 14, wherein the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with all-zeroes.
16. The system of claim 15, wherein replacing the training reference audio signal with all zeroes comprises replacing the training reference audio signal with an all-zero feature of a same length and feature dimension as the corresponding training utterance.
17. The system of claim 15, wherein replacing the training contextual noise signal comprises replacing the training contextual noise signal with an all-zero feature having a predetermined length and a same feature dimension as the corresponding training utterance.
18. The system of claim 15, wherein replacing the training speaker vector comprises replacing the training speaker vector with an all-zero feature with an all-zero vector.
19. The system of claim 14, wherein the signal dropout strategy drops out each training contextual signal by replacing the corresponding contextual signal with a frame-level learned representation.
20. The system of claim 14, wherein the trained contextual frontend processing model comprises: a primary encoder configured to: receive, as input, input speech features corresponding to a target utterance; and generate, as output, a main input encoding; a noise context encoder configured to: receive, as input, a contextual noise signal comprising noise prior to the target utterance; and generate, as output, a contextual noise encoding; and a cross-attention encoder configured to: receive, as input, the main input encoding generated as output from the primary encoder and the contextual noise encoding generated as output from the noise context encoder; and generate, as output, a cross-attention embedding; and a decoder configured to decode the cross-attention embedding into enhanced speech features corresponding to the target utterance.
21. The system of claim 20, wherein the primary encoder is further configured to: receive, as input, reference features corresponding to a reference audio signal; and generate, as output, the main input encoding by processing the input speech features stacked with the reference features.
22. The system of claim 20, wherein the primary encoder is further configured to: receive, as input, a speaker embedding comprising voice characteristics of a target speaker that spoke the target utterance; and generate, as output, the main input encoding by combining the input speech features with the speaker embedding using feature-wise linear modulation (FILM).
23. The system of claim 20, wherein the cross-attention encoder is further configured to: receive, as input, the main input encoding modulated by a speaker embedding using feature-wise linear modulation (FILM), the speaker embedding comprising voice characteristics of a target speaker that spoke the target utterance; and process the main input encoding modulated by the speaker embedding and the contextual noise encoding to generate, as output, the cross-attention embedding.
24. The system of claim 20, wherein: the primary encoder comprises N modulated conformer blocks; the noise context encoder comprises N conformer blocks and executes in parallel with the primary encoder; and the cross-attention encoder comprises M modulated cross-attention conformer blocks.
25. The system of claim 14, wherein the contextual frontend processing model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss.
26. The system of claim 25, wherein the spectral loss is based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask, the ideal ratio mask computed using reverberant speech and reverberant noise.